

Experiment ‘p_4_5_analyse_floats_array’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 2

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_5_analyse_floats_array

Natural language statement: null

Method signature: p_4_5_analyse_floats_array(values: array<real>) returns (average: real, min: real, max: real, range: real)

Ensures

- `average == (sum(values[..]) / values.Length as real)`
- `forall v :: v in values[..] ==> min <= v`
- `forall v :: v in values[..] ==> max >= v`
- `range == max - min`
- `exists v :: v in values[..] && min == v`
- `exists v :: v in values[..] && max == v`

Requires

- `0 < values.Length`

Functional Code Given

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_4_5_analyse_floats_array(values: array<real>) returns (average: real, min: real, max: real, range: real)`

The method should respect the following contract:

`requires 0 < values.Length, ensures average == (sum(values[..]) / values.Length as real),
 ↳ ensures forall v :: v in values[..] ==> min <= v, ensures forall v :: v in values
 ↳ [...] ==> max >= v, ensures range == max - min, ensures exists v :: v in values[..]
 ↳ && min == v, ensures exists v :: v in values[..] && max == v`

The contract uses the following dafny code:

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
 ↳ surround it. Do not show the signature nor contract. You must not use 'assume' nor
 ↳ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnM9Bw34NxU9eIMyDa18zxnByCahV

Model: gpt-5.5-2026-04-23

Created at: 1780655577

Finish reason: stop

Usage: 558 tokens in, and 8302 tokens out

```
{
  var n := values.Length;

  var total := 0.0;
  var i := n;
  while 0 < i
    invariant 0 <= i <= n
```

```

    invariant total == sum(values[i..n])
  {
    i := i - 1;
    assert 0 <= i < n;
    assert total == sum(values[i + 1..n]);
    assert |values[i..n]| > 0;
    assert (values[i..n])[0] == values[i];
    assert (values[i..n])[1..] == values[i + 1..n];
    assert sum(values[i..n]) == values[i] + sum(values[i + 1..n]);
    total := values[i] + total;
    assert total == sum(values[i..n]);
  }
  assert i == 0;
  assert values[0..n] == values[..];
  assert total == sum(values[..]);

  min := values[0];
  max := values[0];
  var minIndex := 0;
  var maxIndex := 0;
  i := 1;
  while i < n
    invariant 1 <= i <= n
    invariant forall k: int :: 0 <= k < i ==> min <= values[k]
    invariant forall k: int :: 0 <= k < i ==> max >= values[k]
    invariant 0 <= minIndex < i
    invariant min == values[minIndex]
    invariant 0 <= maxIndex < i
    invariant max == values[maxIndex]
  {
    var x := values[i];
    if x < min {
      min := x;
      minIndex := i;
    }
    if x > max {
      max := x;
      maxIndex := i;
    }
    i := i + 1;
  }
  assert i == n;
  assert forall k: int :: 0 <= k < n ==> min <= values[k];
  assert forall k: int :: 0 <= k < n ==> max >= values[k];

  average := total / (n as real);
  range := max - min;
  assert n == values.Length;
  assert average == sum(values[..]) / (values.Length as real);

  assert forall v: real :: v in values[..] ==> min <= v by {

```

```

    if v in values[..] {
      assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
      var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
      assert |values[..]| == n;
      assert values[..][k] == values[k];
      assert v == values[k];
      assert min <= values[k];
    }
  }
  assert forall v: real :: v in values[..] ==> max >= v by {
    if v in values[..] {
      assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
      var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
      assert |values[..]| == n;
      assert values[..][k] == values[k];
      assert v == values[k];
      assert max >= values[k];
    }
  }

  assert 0 <= minIndex < |values[..]|;
  assert values[..][minIndex] == min;
  assert min in values[..];
  assert exists v: real :: v in values[..] && min == v;

  assert 0 <= maxIndex < |values[..]|;
  assert values[..][maxIndex] == max;
  assert max in values[..];
  assert exists v: real :: v in values[..] && max == v;
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_5_analyse_floats_array/program.dfy(73,7): Error: unresolved identifier:

```

    ↪ v
    |
73 | if v in values[..] {
    | ^

```

problem_data/p_4_5_analyse_floats_array/program.dfy(74,74): Error: unresolved identifier:

```

    ↪ v
    |
74 | assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
    | ^

```

problem_data/p_4_5_analyse_floats_array/program.dfy(75,64): Error: unresolved identifier:

```

    ↪ v
    |
75 | var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;

```

```

    | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(78,13): Error: unresolved identifier:
    ↪ v
    |
78 | assert v == values[k];
    | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(83,7): Error: unresolved identifier:
    ↪ v
    |
83 | if v in values[..] {
    | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(84,74): Error: unresolved identifier:
    ↪ v
    |
84 | assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
    | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(85,64): Error: unresolved identifier:
    ↪ v
    |
85 | var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
    | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(88,13): Error: unresolved identifier:
    ↪ v
    |
88 | assert v == values[k];
    | ^

8 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnMBykXOfm2DvAaqbWCCKlezP1kMg

Model: gpt-5.5-2026-04-23

Created at: 1780655750

Finish reason: stop

Usage: 2226 tokens in, and 1894 tokens out

```

{
  var n := values.Length;

  var total := 0.0;

```

```

var i := n;
while 0 < i
  invariant 0 <= i <= n
  invariant total == sum(values[i..n])
{
  i := i - 1;
  assert 0 <= i < n;
  assert total == sum(values[i + 1..n]);
  assert |values[i..n]| > 0;
  assert (values[i..n])[0] == values[i];
  assert (values[i..n])[1..] == values[i + 1..n];
  assert sum(values[i..n]) == values[i] + sum(values[i + 1..n]);
  total := values[i] + total;
  assert total == sum(values[i..n]);
}
assert i == 0;
assert values[0..n] == values[..];
assert total == sum(values[..]);

min := values[0];
max := values[0];
var minIndex := 0;
var maxIndex := 0;
i := 1;
while i < n
  invariant 1 <= i <= n
  invariant forall k: int :: 0 <= k < i ==> min <= values[k]
  invariant forall k: int :: 0 <= k < i ==> max >= values[k]
  invariant 0 <= minIndex < i
  invariant min == values[minIndex]
  invariant 0 <= maxIndex < i
  invariant max == values[maxIndex]
{
  var x := values[i];
  if x < min {
    min := x;
    minIndex := i;
  }
  if x > max {
    max := x;
    maxIndex := i;
  }
  i := i + 1;
}
assert i == n;
assert forall k: int :: 0 <= k < n ==> min <= values[k];
assert forall k: int :: 0 <= k < n ==> max >= values[k];

average := total / (n as real);
range := max - min;
assert n == values.Length;

```

```

assert average == sum(values[..]) / (values.Length as real);

assert forall v: real | v in values[..]
  ensures min <= v
{
  assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
  var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
  assert |values[..]| == n;
  assert (values[..])[k] == values[k];
  assert v == values[k];
  assert min <= values[k];
}
assert forall v: real | v in values[..]
  ensures max >= v
{
  assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
  var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
  assert |values[..]| == n;
  assert (values[..])[k] == values[k];
  assert v == values[k];
  assert max >= values[k];
}

assert 0 <= minIndex < |values[..]|;
assert (values[..])[minIndex] == min;
assert min in values[..];
assert exists v: real :: v in values[..] && min == v;

assert 0 <= maxIndex < |values[..]|;
assert (values[..])[maxIndex] == max;
assert max in values[..];
assert exists v: real :: v in values[..] && max == v;
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```
problem_data/p_4_5_analyse_floats_array/program.dfy(102,0): Error: invalid
```

```
  ↳ UnaryExpression
```

```

|
102 | }
| ^

```

```
problem_data/p_4_5_analyse_floats_array/program.dfy(101,54): Error: expected either 'by'
```

```
  ↳ or a semicolon following the assert expression
```

```

|
101 | assert exists v: real :: v in values[..] && max == v;
| ^

```

```
problem_data/p_4_5_analyse_floats_array/program.dfy(101,54): Error: expected either 'by'
  ↳ or a semicolon following the assert expression
  |
101 | assert exists v: real :: v in values[..] && max == v;
  | ^
```

3 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature
 ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnMCiRHOL2DrFiPseTRFDQk5jMVN1

Model: gpt-5.5-2026-04-23

Created at: 1780655796

Finish reason: stop

Usage: 3094 tokens in, and 2444 tokens out

```
{
  var n := values.Length;

  var total := 0.0;
  var i := n;
  while 0 < i
    invariant 0 <= i <= n
    invariant total == sum(values[i..n])
  {
    i := i - 1;
    assert 0 <= i < n;
    assert total == sum(values[i + 1..n]);
    assert |values[i..n]| > 0;
    assert (values[i..n])[0] == values[i];
    assert (values[i..n])[1..] == values[i + 1..n];
    assert sum(values[i..n]) == values[i] + sum(values[i + 1..n]);
    total := values[i] + total;
    assert total == sum(values[i..n]);
  }
  assert i == 0;
  assert values[0..n] == values[..];
  assert total == sum(values[..]);

  min := values[0];
  max := values[0];
  var minIndex := 0;
  var maxIndex := 0;
  i := 1;
  while i < n
    invariant 1 <= i <= n
```



```

invariant forall k: int :: 0 <= k < i ==> min <= values[k]
invariant forall k: int :: 0 <= k < i ==> max >= values[k]
invariant 0 <= minIndex < i
invariant min == values[minIndex]
invariant 0 <= maxIndex < i
invariant max == values[maxIndex]
{
  var x := values[i];
  if x < min {
    min := x;
    minIndex := i;
  }
  if x > max {
    max := x;
    maxIndex := i;
  }
  i := i + 1;
}
assert i == n;
assert forall k: int :: 0 <= k < n ==> min <= values[k];
assert forall k: int :: 0 <= k < n ==> max >= values[k];

average := total / (n as real);
range := max - min;
assert n == values.Length;
assert average == sum(values[..]) / (values.Length as real);

forall v: real | v in values[..]
  ensures min <= v
{
  assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
  var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
  assert |values[..]| == n;
  assert (values[..])[k] == values[k];
  assert v == values[k];
  assert min <= values[k];
}
forall v: real | v in values[..]
  ensures max >= v
{
  assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
  var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
  assert |values[..]| == n;
  assert (values[..])[k] == values[k];
  assert v == values[k];
  assert max >= values[k];
}

assert 0 <= minIndex < |values[..]|;
assert (values[..])[minIndex] == min;
assert min in values[..];

```

```

    assert exists v: real :: (v in values[..]) && min == v;

    assert 0 <= maxIndex < |values[..]|;
    assert (values[..])[maxIndex] == max;
    assert max in values[..];
    assert exists v: real :: (v in values[..]) && max == v;
}

```

Final Program

```

function sum(values: seq<real>): real
{
    if |values| == 0 then 0.0
    else values[0] + sum(values[1..])
}

method p_4_5_analyse_floats_array(values: array<real>) returns (average: real, min: real,
    ↪ max: real, range: real)
    requires 0 < values.Length
    ensures average == (sum(values[..]) / values.Length as real)
    ensures forall v :: v in values[..] ==> min <= v
    ensures forall v :: v in values[..] ==> max >= v
    ensures range == max - min
    ensures exists v :: v in values[..] && min == v
    ensures exists v :: v in values[..] && max == v
{
    var n := values.Length;

    var total := 0.0;
    var i := n;
    while 0 < i
        invariant 0 <= i <= n
        invariant total == sum(values[i..n])
    {
        i := i - 1;
        assert 0 <= i < n;
        assert total == sum(values[i + 1..n]);
        assert |values[i..n]| > 0;
        assert (values[i..n])[0] == values[i];
        assert (values[i..n])[1..] == values[i + 1..n];
        assert sum(values[i..n]) == values[i] + sum(values[i + 1..n]);
        total := values[i] + total;
        assert total == sum(values[i..n]);
    }
    assert i == 0;
    assert values[0..n] == values[..];
    assert total == sum(values[..]);

    min := values[0];
    max := values[0];

```

```

var minIndex := 0;
var maxIndex := 0;
i := 1;
while i < n
  invariant 1 <= i <= n
  invariant forall k: int :: 0 <= k < i ==> min <= values[k]
  invariant forall k: int :: 0 <= k < i ==> max >= values[k]
  invariant 0 <= minIndex < i
  invariant min == values[minIndex]
  invariant 0 <= maxIndex < i
  invariant max == values[maxIndex]
{
  var x := values[i];
  if x < min {
    min := x;
    minIndex := i;
  }
  if x > max {
    max := x;
    maxIndex := i;
  }
  i := i + 1;
}
assert i == n;
assert forall k: int :: 0 <= k < n ==> min <= values[k];
assert forall k: int :: 0 <= k < n ==> max >= values[k];

average := total / (n as real);
range := max - min;
assert n == values.Length;
assert average == sum(values[..]) / (values.Length as real);

forall v: real | v in values[..]
  ensures min <= v
{
  assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
  var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
  assert |values[..]| == n;
  assert (values[..])[k] == values[k];
  assert v == values[k];
  assert min <= values[k];
}
forall v: real | v in values[..]
  ensures max >= v
{
  assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
  var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
  assert |values[..]| == n;
  assert (values[..])[k] == values[k];
  assert v == values[k];
  assert max >= values[k];
}

```

```
}  
  
assert 0 <= minIndex < |values[..]|;  
assert (values[..])[minIndex] == min;  
assert min in values[..];  
assert exists v: real :: (v in values[..]) && min == v;  
  
assert 0 <= maxIndex < |values[..]|;  
assert (values[..])[maxIndex] == max;  
assert max in values[..];  
assert exists v: real :: (v in values[..]) && max == v;  
}
```

Total Token Usage

Input tokens: 5878

Output tokens: 12640

Reasoning tokens: 9834

Sum of 'total tokens': 18518

Experiment Timings

Overall Experiment started at 1780655576606, ended at 1780655867699, lasting 291093ms (291.09 seconds)

Iteration #1 started at 1780655576606, ended at 1780655749995, lasting 173389ms (173.39 seconds)

Iteration #2 started at 1780655749996, ended at 1780655796534, lasting 46538ms (46.54 seconds)

Iteration #3 started at 1780655796534, ended at 1780655867699, lasting 71165ms (71.17 seconds)